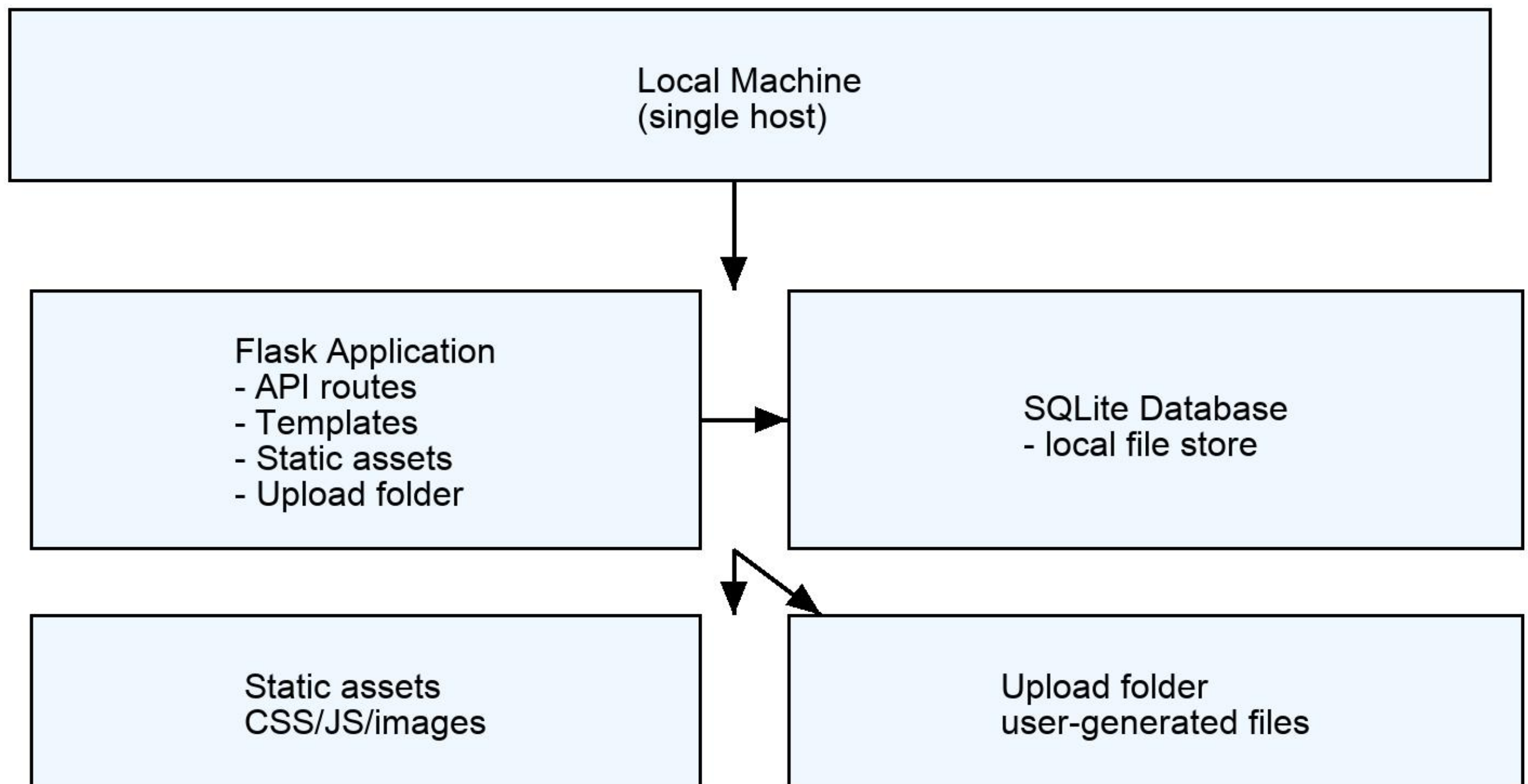


Screenshot 1: Current Local Architecture Diagram

Current Local Architecture



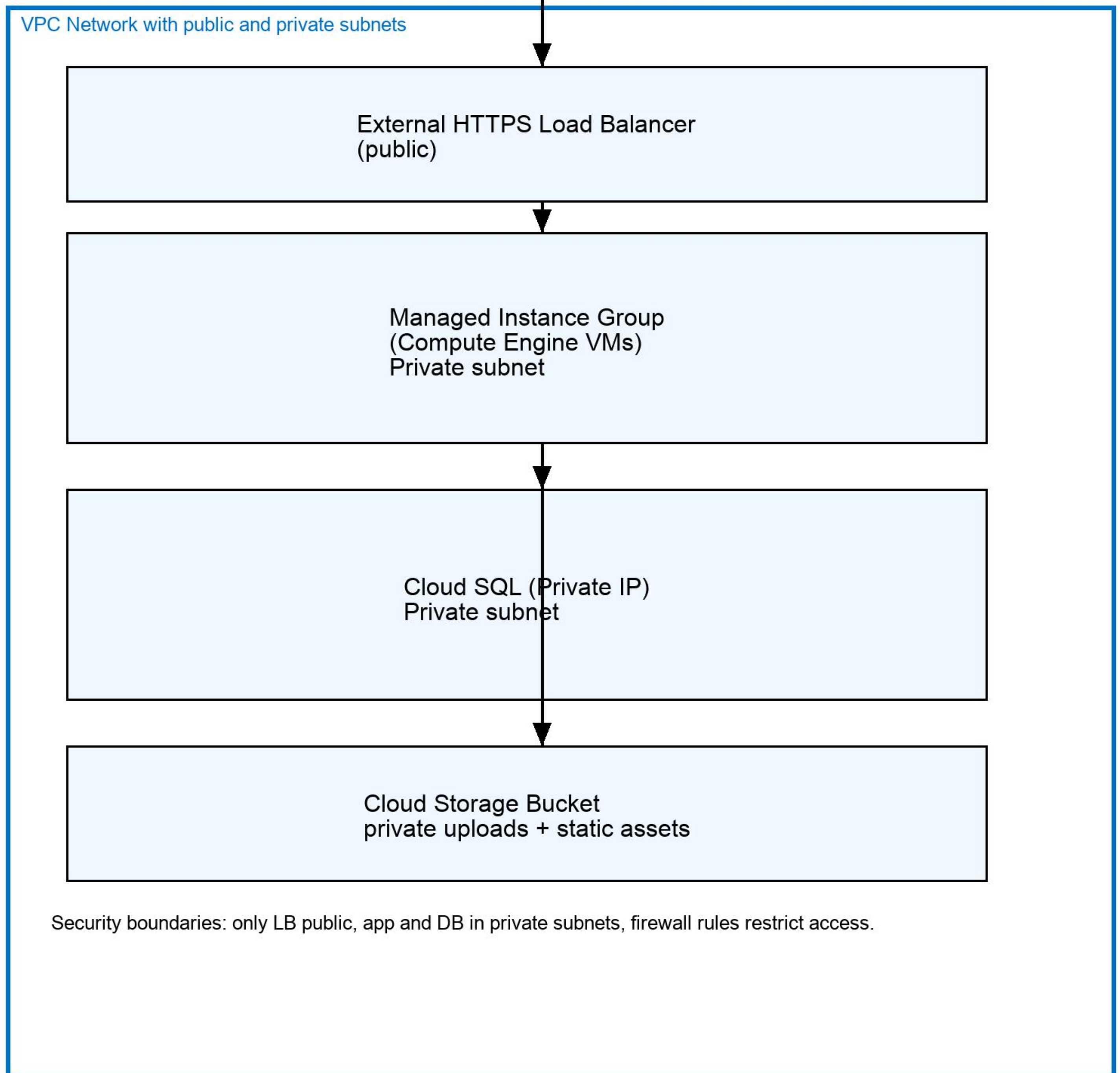
Component classification: Compute = Flask app, Database = SQLite, Storage = static/upload files, Networking = local interface

Caption:

This diagram shows the single-host CloudNotes deployment with the Flask application, SQLite database, static assets, templates, uploads, and local networking all contained on one machine.

Screenshot 2: Proposed Cloud Architecture Diagram

Proposed Google Cloud Architecture



Caption:

This proposed GCP architecture puts the public ingress on an HTTPS Load Balancer, and places the application tier, database tier, and storage inside private networks with firewall protections.

Screenshot 3: Component-to-Service Mapping Table

Component-to-Service Mapping

Component	Service Category		Proposed GCP Service	Justification
Flask App	Compute		Compute Engine	Runs containerized application workloads on
SQLite DB	Database		Cloud SQL	managed VMs in a private subnet. Replaces local file database with managed relational database with
Static assets	Storage		Cloud Storage	storage, enabling durability and HA. Stores web assets and uploads with durable object
Upload folder	Storage		Cloud Storage	storage and IAM control. Provides scalable file persistence separated from
API routes	Networking		HTTPS Load Balancer	compute lifecycle. Fronts private compute with a managed public ingress point and traffic distribution.

Caption:

This table maps each CloudNotes component to its service category and proposed GCP service with engineering justification for each choice.

Screenshot 4: Distributed Systems Risk Analysis

Distributed Systems Risk Analysis

Scenario A: DB zone failure

A private Cloud SQL zone outage makes the application lose writes and reads. CAP: consistency prioritized by managed SQL, so availability drops during failover. Mitigation: use regional Cloud SQL, multi-AZ failover, and replicas to improve availability without sacrificing data consistency.

Scenario B: 10× traffic spike

A single VM cannot absorb sudden demand and becomes overloaded. Autoscaling and load balancing distribute requests across multiple instances, scaling compute capacity while preserving responsiveness.

Network partition

A VPC or backend failure can isolate compute from DB/storage. Mitigation: private service connectivity, firewall health checks, and retry logic reduce impact from transient partitions.

Caption:

This analysis covers database zone failure, traffic spikes, and network partition risks with CAP reasoning and mitigation strategies for a cloud deployment.

Screenshot 5: Architecture Decision Record (ADR)

Architecture Decision Record (ADR)

Decision: Use Compute Engine with private managed instance group.

Context:

Context: The app is a small Flask service currently on a single local host.

Alternatives:

Alternatives: Cloud Run, GKE, App Engine.

Trade-off:

Trade-off: Compute Engine gives explicit networking and private subnet control at the cost of more infrastructure management.

Decision: Replace SQLite with Cloud SQL.

Context:

Context: SQLite is local and not durable across cloud VMs.

Alternatives:

Alternatives: keep SQLite on the VM, use Cloud Spanner, use Firestore.

Trade-off:

Trade-off: Cloud SQL incurs managed service cost but provides backups, private networking, and HA.

Decision: Store uploads and static assets in Cloud Storage.

Context:

Context: Local files on the VM do not scale or survive instance replacement.

Alternatives:

Alternatives: keep file storage on VM disk or use a shared filesystem.

Trade-off:

Trade-off: Cloud Storage is highly durable and decouples storage from compute, while requiring application changes for object access.

Caption:

This ADR records the major design decisions, context, alternatives considered, and trade-offs for the chosen cloud architecture.